

4. Ocena wyników modelu i optymalizacja

Link do google collab:

<https://colab.research.google.com/drive/1ccxw5PQN3Mlzvalsz5BUrfQQvw1lvYa?usp=sharing>

Opis poszczególnych bloków:

```
) from google.colab import drive
drive.mount('/content/drive')
```

```
• Mounted at /content/drive
```

```
import torch
print("Torch version:", torch.__version__)
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "brak")
```

```
Torch version: 2.10.0+cu128
GPU: Tesla T4
```

```
!pip install frEIA h5py
```

Instalacja bibliotek, podpięcie dysku z danymi

```

import torch
import torch.nn as nn
import h5py
from torch.utils.data import TensorDataset, DataLoader
import FrEIA.framework as Ff
import FrEIA.modules as Fm

H5_FILE = "/content/drive/MyDrive/Colab Notebooks/stellar_training_data.h5"
BATCH_SIZE = 32
EPOCHS = 100
LR = 1e-4

INPUT_DIM = 8575
COND_DIM = 3

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Używane urządzenie: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU'}")

```

Używane urządzenie: Tesla T4

```

def subnet_fc(in_dim, out_dim):
    return nn.Sequential(
        nn.Linear(in_dim, 512),
        nn.ReLU(),
        nn.Linear(512, out_dim)
    )

def build_cinn(input_dim, cond_dim, n_blocks=6):
    inn = Ff.SequenceINN(input_dim)

    for _ in range(n_blocks):
        inn.append(
            Fm.AllInOneBlock,
            cond=0,
            cond_shape=(cond_dim,),
            subnet_constructor=subnet_fc,
            affine_clamping=2.0,
            permute_soft=False,
        )

    return inn

```

```

def cinn_loss(z, log_jac_det, input_dim):
    z_term = 0.5 * torch.sum(z**2, dim=1)
    loss = torch.mean(z_term - log_jac_det)
    return loss / input_dim

```

Kod implementuje w PyTorch i FrEIA warunkową sieć odwracalną (cINN), która modeluje rozkład widm gwiazd (8575 wymiarów) w zależności od 3 parametrów fizycznych. Wykorzystując 6 bloków transformacji afinicznej z podsieciami o rozmiarze 512 neuronów, model uczy się mapować dane na szum Gaussa poprzez optymalizację log-wiarygodności (NLL) z uwzględnieniem wyznacznika Jakobianu. Całość pozwala na stabilną, odwracalną syntezę i rekonstrukcję sygnałów na procesorze graficznym Tesla T4.

```

def load_data(h5_path):
    with h5py.File(h5_path, "r") as f:
        print("Dostępne klucze:", list(f.keys()))
        x = torch.tensor(f["spectra"][:], dtype=torch.float32)
        c = torch.tensor(f["stellar_labels"][:], dtype=torch.float32)

        assert not torch.isnan(x).any(), "NaN w spektrach!"
        assert not torch.isinf(x).any(), "Inf w spektrach!"
        assert not torch.isnan(c).any(), "NaN w labelach!"
        assert not torch.isinf(c).any(), "Inf w labelach!"

        x_mean = x.mean(0, keepdim=True)
        x_std = x.std(0, keepdim=True).clamp_min(1e-6)
        x = (x - x_mean) / x_std

        c = (c - c.mean(0, keepdim=True)) / c.std(0, keepdim=True).clamp_min(1e-6)
    return x, c

```

```

print("Wczytywanie danych...")
x, c = load_data(H5_FILE)

dataset = TensorDataset(x, c)
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
print(f"Dane wczytane: {x.shape[0]} widm, dim={x.shape[1]}, warunki dim={c.shape[1]}")

```

```

Wczytywanie danych...
Dostępne klucze: ['spectra', 'star_ids', 'stellar_labels']
Dane wczytane: 10675 widm, dim=8575, warunki dim=3

```

```

print("Budowanie modelu...")
model = build_cinn(INPUT_DIM, COND_DIM).to(DEVICE)

optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', patience=10, factor=0.5
)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model ma {total_params:,} parametrów.")

```

```

Budowanie modelu...
Model ma 39,678,696 parametrów.

```

Skrypt realizuje wczytanie, walidację i standaryzację 10 675 widm gwiazdowych, tworząc wydajny DataLoader. Następnie inicjalizuje na GPU model cINN o pojemności 40 mln parametrów, konfigurując optymalizator Adam oraz scheduler ReduceLROnPlateau do stabilnego modelowania złożonych struktur spektralnych.

```

print(f"Start treningu ({EPOCHS} epok)...")
model.train()

for epoch in range(1, EPOCHS + 1):
    total_loss = 0.0
    total_z_sq = 0.0
    total_logdet = 0.0

    for batch_x, batch_c in loader:
        batch_x = batch_x.to(DEVICE)
        batch_c = batch_c.to(DEVICE)

        optimizer.zero_grad()

        z, log_jac_det = model(batch_x, c=[batch_c])

        loss = cinn_loss(z, log_jac_det, INPUT_DIM)
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=10.0)
        optimizer.step()

        total_loss += loss.item()
        with torch.no_grad():
            z_sq = 0.5 * (z**2).sum(dim=1).mean()
            total_z_sq += z_sq.item()
            total_logdet += (-log_jac_det).mean().item()

    avg_loss = total_loss / len(loader)
    avg_z_sq = total_z_sq / len(loader)
    avg_logdet = total_logdet / len(loader)

    print(f"Epoch {epoch:3d}/{EPOCHS} | Loss: {avg_loss:.4f} "
          f"z²: {avg_z_sq:.2f}, -log|det|: {avg_logdet:.2f}")

    scheduler.step(avg_loss)

```

Dostarczony fragment kodu obrazuje główną pętlę treningową modelu cINN, która realizuje proces uczenia przez zadaną liczbę stu epok. Wewnątrz każdej iteracji dane są przetwarzane w paczkach, gdzie najpierw następuje wyzerowanie gradientów optymalizatora, a następnie wykonanie przejścia w przód przez sieć w celu uzyskania reprezentacji utajonej oraz logarytmu wyznacznika Jakobianu przy uwzględnieniu warunków gwiazdowych. Na tej podstawie obliczana jest zdefiniowana wcześniej funkcja straty, która poprzez mechanizm propagacji wstecznej dostarcza informacji niezbędnych do aktualizacji wag modelu. Istotnym elementem stabilizującym proces uczenia jest zastosowanie przycinania gradientów (gradient clipping) do wartości 10.0, co zapobiega gwałtownym wahaniom parametrów sieci i sprzyja stabilności numerycznej odwracalnych bloków. Kod dodatkowo monitoruje szczegółowe statystyki dotyczące kwadratu normy wektora utajonego oraz zmiany objętości transformacji, wyświetlając po każdej epoce średnie wartości straty oraz jej komponentów, co pozwala na bieżąco śledzić jakość mapowania danych na rozkład normalny. Na zakończenie każdej epoki planista tempa uczenia dokonuje ewentualnej korekty parametru uczenia, reagując na poziom osiągniętej straty średniej, co domyka pełny cykl optymalizacji sieci i zapewnia płynne schodzenie do minimum funkcji celu.

```
*** Epoch 100/100 | Loss: -3.0456 (z²: 7926.16, -log|det|: -34041.85)
```

```
▶ save_path = "/content/drive/MyDrive/Colab Notebooks/cinn_model_ep100.pt"
  torch.save(model.state_dict(), save_path)
  print(f"Model zapisany jako {save_path}")
```

```
... Model zapisany jako /content/drive/MyDrive/Colab Notebooks/cinn_model_ep100.pt
```

```
def sample(model, c_sample=None):
    model.eval()
    if c_sample is None:
        c_sample = torch.randn(1, COND_DIM).to(DEVICE)
    z = torch.randn(1, INPUT_DIM).to(DEVICE)
    with torch.no_grad():
        x_gen, _ = model(z, c=[c_sample], rev=True)
    return x_gen
```

```
x_gen = sample(model)
print("Wygenerowana próbka:", x_gen.shape)
```

```
Wygenerowana próbka: torch.Size([1, 8575])
```

```
import torch.nn.functional as F
from torch.utils.data import random_split, DataLoader
```

```
VAL_FRAC = 0.1
n_val = max(1, int(len(dataset) * VAL_FRAC))
n_train = len(dataset) - n_val
```

```
train_ds, val_ds = random_split(dataset, [n_train, n_val], generator=torch.Generator().manual_seed(42))
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False)
```

```
print(f"Train size: {len(train_ds)} | Val size: {len(val_ds)}")
```

```
Train size: 9608 | Val size: 1067
```

Przedstawiony fragment kodu rozpoczyna się od zapisu wyuczonych wag modelu do pliku na Dysku Google, co pozwala na późniejszą reutilizację sieci bez konieczności ponownego trenowania. Następnie zdefiniowana zostaje funkcja `sample`, która wykorzystuje unikalną cechę sieci odwracalnych, czyli możliwość generowania nowych danych poprzez przekształcenie losowego szumu gaussowskiego z z powrotem w przestrzeń widm przy użyciu trybu `rev=True`. Funkcja ta umożliwia syntezę sztucznych widm gwiazdowych o wymiarze 8575 punktów pod warunkiem zadanych parametrów fizycznych, co zostaje potwierdzone wygenerowaniem przykładowej próbki o oczekiwanym kształcie. Ostatnia sekcja kodu odpowiada za podział zbioru danych na część treningową i walidacyjną w stosunku 9 do 1, co skutkuje wyodrębnieniem 1067 próbek do niezależnych testów. Proces ten kończy się utworzeniem obiektu `val_loader`, który służy do obiektywnej oceny jakości modelu na danych, których sieć nie widziała podczas procesu optymalizacji, zapewniając solidną podstawę do dalszej weryfikacji zdolności generalizacji modelu.

```

def reconstruction_test(model, loader, n_batches=5):
    model.eval()
    mses = []
    max_abs_errs = []

    with torch.no_grad():
        for i, (batch_x, batch_c) in enumerate(loader):
            if i >= n_batches:
                break

            batch_x = batch_x.to(DEVICE)
            batch_c = batch_c.to(DEVICE)

            z, log_jac_det = model(batch_x, c=[batch_c])
            x_rec, _ = model(z, c=[batch_c], rev=True)

            mse = F.mse_loss(x_rec, batch_x).item()
            max_err = (x_rec - batch_x).abs().max().item()

            mses.append(mse)
            max_abs_errs.append(max_err)

    print(f"Reconstruction MSE: {sum(mses)/len(mses):.6e}")
    print(f"Max abs error: {sum(max_abs_errs)/len(max_abs_errs):.6e}")

reconstruction_test(model, val_loader, n_batches=5)

```

```

*** Reconstruction MSE: 3.901412e-14
    Max abs error: 1.230240e-05

```

```

def latent_sanity_test(model, loader, n_batches=5):
    model.eval()
    z_means, z_stds, logdets = [], [], []

    with torch.no_grad():
        for i, (batch_x, batch_c) in enumerate(loader):
            if i >= n_batches:
                break

            batch_x = batch_x.to(DEVICE)
            batch_c = batch_c.to(DEVICE)

            z, log_jac_det = model(batch_x, c=[batch_c])

            z_means.append(z.mean().item())
            z_stds.append(z.std().item())
            logdets.append(log_jac_det.mean().item())

    print(f"Mean(z): {sum(z_means)/len(z_means):.4f}")
    print(f"Std(z): {sum(z_stds)/len(z_stds):.4f}")
    print(f"Mean log|det|: {sum(logdets)/len(logdets):.4f}")

latent_sanity_test(model, val_loader, n_batches=5)

```

```

*** Mean(z): 0.0140
    Std(z): 0.9252
    Mean log|det|: 33851.0383

```

```

def reconstruction_test(model, loader, n_batches=5):
    model.eval()
    mses = []

```

```

▶ def conditional_sampling_test(model, loader, n_conditions=3, n_samples=8):
    model.eval()
    batch_x, batch_c = next(iter(loader))
    batch_c = batch_c.to(DEVICE)

    idxs = torch.linspace(0, len(batch_c) - 1, steps=min(n_conditions, len(batch_c))).long()

    with torch.no_grad():
        for j, idx in enumerate(idxs):
            c0 = batch_c[idx:idx+1]
            c_rep = c0.repeat(n_samples, 1)
            z = torch.randn(n_samples, INPUT_DIM, device=DEVICE)
            x_gen, _ = model(z, c=[c_rep], rev=True)

            sample_std = x_gen.std(dim=0).mean().item()
            sample_mean = x_gen.mean().mean().item()

            print(f"Condition {j+1}: generated batch mean={sample_mean:.4f}, mean std across pixels={:

    conditional_sampling_test(model, val_loader, n_conditions=3, n_samples=8)

```

```

*** Condition 1: generated batch mean=-0.0476, mean std across pixels=0.1346
    Condition 2: generated batch mean=-0.0926, mean std across pixels=0.1001
    Condition 3: generated batch mean=-0.0731, mean std across pixels=0.1153

```

```

def interpolation_test(model, loader, n_steps=5):
    model.eval()
    batch_x, batch_c = next(iter(loader))
    c1 = batch_c[0:1].to(DEVICE)
    c2 = batch_c[1:2].to(DEVICE)

    z = torch.randn(1, INPUT_DIM, device=DEVICE)
    for a in torch.linspace(0, 1, steps=n_steps):
        c_mix = (1 - a) * c1 + a * c2
        with torch.no_grad():
            x_gen, _ = model(z, c=[c_mix], rev=True)
            print(f"alpha={a.item():.2f} | generated shape={tuple(x_gen.shape)} | mean={x_gen.mean().it

    interpolation_test(model, val_loader, n_steps=5)

```

```

alpha=0.00 | generated shape=(1, 8575) | mean=0.0419
alpha=0.25 | generated shape=(1, 8575) | mean=0.0433
alpha=0.50 | generated shape=(1, 8575) | mean=0.0448
alpha=0.75 | generated shape=(1, 8575) | mean=0.0463
alpha=1.00 | generated shape=(1, 8575) | mean=0.0479

```

```

def real_vs_generated_stats(model, loader, n_samples=256):
    model.eval()
    batch_x, batch_c = next(iter(loader))
    batch_x = batch_x.to(DEVICE)
    batch_c = batch_c.to(DEVICE)

    n = min(n_samples, batch_x.shape[0])
    x_real = batch_x[:n]
    c_real = batch_c[:n]

    with torch.no_grad():
        z = torch.randn(n, INPUT_DIM, device=DEVICE)
        x_fake, _ = model(z, c=[c_real], rev=True)

    real_mean = x_real.mean().item()
    real_std = x_real.std().item()
    fake_mean = x_fake.mean().item()
    fake_std = x_fake.std().item()

    print(f"Real mean/std: {real_mean:.4f} / {real_std:.4f}")
    print(f"Fake mean/std: {fake_mean:.4f} / {fake_std:.4f}")

    real_vs_generated_stats(model, val_loader, n_samples=256)

```

```

... Real mean/std: 0.0600 / 1.1740
Fake mean/std: -0.0346 / 0.4593

```